LAPORAN PRAKTIKUM
MODUL VI - POLYMORPHISM

Diajukan untuk memenuhi salah satu tugas praktikum
Mata kuliah Pemrograman Berorientasi Objek



Disusun Oleh:
Radithya Mahesa Syabil (251511025)
Jurusan Teknik Komputer dan Informatika

Program Studi D-3 Teknik Informatika
Politeknik Negeri Bandung
2026

BAGIAN A PRAKTIKUM I: ANOTHER TYPE OF EMPLOYEE

A.1 Step 1 Petakan Hierarchy dan Reference Type

```
StaffMember
├── Volunteer
└── Employee
    ├── Executive
    └── Hourly
        └── Commission // akan Anda tambahkan
```

- Tandai mana class abstract, mana concrete class, dan method mana yang dioverride (fokuskan perhatian pada method pay()).

  Jawaban:

  StaffMember adalah abstract class (memiliki abstract method pay()). Volunteer, Employee, Executive, Hourly, dan Commission adalah concrete class. Method pay() di-override oleh Volunteer, Employee, Executive, Hourly, dan Commission, sedangkan StaffMember hanya mendeklarasikannya sebagai abstract.

- Checkpoint 1: Apa tipe array pada Staff? Mengapa array tersebut dapat berisi Executive, Employee, Hourly, Volunteer, dan nanti Commission?

  Jawaban:

  Tipe array pada Staff adalah StaffMember[] (private StaffMember[] staffList). Array tersebut dapat berisi Executive, Employee, Hourly, Volunteer, dan nanti Commission karena semuanya merupakan subclass dari StaffMember (relasi IS-A), sehingga setiap object tersebut dapat dirujuk oleh reference bertipe StaffMember.

A.2 Step 2 Jalankan Baseline Payroll

```
private StaffMember[] staffList;

...

for (int count = 0; count < staffList.length; count++) {
    System.out.println(staffList[count]);
    amount = staffList[count].pay(); // polymorphic call
    ...
}
```


Gambar 1. Output baseline payroll Firm.java

Gambar 2. Output baseline payroll Firm.java (lanjutan)

- Catat minimal tiga jenis object berbeda yang berada di staffList.

  Jawaban:

  Object berbeda yang ada di staffList: Executive (Sam), Employee (Carla dan Woody), Hourly (Diane), serta Volunteer (Norm dan Cliff).

- Catat nilai pay() yang dihasilkan oleh Employee, Hourly, dan Volunteer.

  Jawaban:

Employee menghasilkan Paid: 1246.15 (Carla) dan 1169.23 (Woody), Executive (Sam) menghasilkan 2923.07, Hourly (Diane, 40 jam) menghasilkan 422.0, sedangkan Volunteer (Norm dan Cliff) tidak dibayar sehingga pay() bernilai 0.0 dan program hanya mencetak Thanks!.

- Checkpoint 2: Reference yang dipakai oleh loop adalah StaffMember, tetapi implementation pay() yang berjalan tidak selalu StaffMember. Apa yang menentukan implementation yang dipilih?

  Jawaban:

  Implementation pay() yang dijalankan ditentukan oleh actual object type, yaitu class dari object yang sebenarnya dirujuk oleh staffList[count] saat runtime, bukan oleh reference type StaffMember.

A.3 Step 3 Prediksi Sebelum Membuat Commission

- Tulis state baru yang hanya dibutuhkan Commission.

  Jawaban:

  State baru yang hanya dibutuhkan Commission adalah totalSales (double) dan commissionRate (double).

- Behavior Hourly apa yang masih dibutuhkan Commission?

  Jawaban:

  Commission masih membutuhkan behavior Hourly berupa addHours(), pay() (perhitungan upah per jam), dan toString().

- Apakah Commission perlu menulis ulang mekanisme perhitungan upah per jam dari nol? Jelaskan prediksi Anda.

  Jawaban:

  Tidak perlu. Perhitungan upah per jam sudah ada di Hourly.pay(), sehingga Commission cukup memanggil super.pay() lalu menambahkan komisi (totalSales * commissionRate). Dengan begitu Commission hanya menambah logika yang memang baru.

A.4 Step 4 Implementasikan Commission

Potongan kode Commission.java:

```
public class Commission extends Hourly {
    private double totalSales;
    private double commissionRate;

    public Commission(String name, String address, String phone,
                      String ssn, double rate,
                      double commissionRate) {
        super(name, address, phone, ssn, rate);
        this.commissionRate = commissionRate;
        this.totalSales = 0;
    }

    public void addSales(double sales) {
        totalSales += sales;
    }

    @Override
    public double pay() {
        double payment = super.pay()
                       + (totalSales * commissionRate);
        totalSales = 0;
        return payment;
    }

    @Override
    public String toString() {
        String result = super.toString();
        result += "\nTotal Sales: " + totalSales;
        return result;
    }
}
```

- Checkpoint 3: Mengapa pay() pada Commission sebaiknya memanggil super.pay() daripada menghitung kembali hoursWorked? Hubungkan dengan encapsulation dan reuse behavior.

  Jawaban:

  Memanggil super.pay() menjaga encapsulation karena hoursWorked bersifat private di Hourly sehingga Commission tidak dapat (dan tidak perlu) mengaksesnya langsung, termasuk reset hoursWorked menjadi 0 yang sudah dilakukan di dalam Hourly.pay().

A.5 Step 5 Tambahkan Dua Commission ke Staff

Potongan kode pada Staff.java:

```
Commission c1 = new Commission("Andi", "101 First St.",
    "555-1111", "111-11-1111", 6.25, 0.20);
c1.addHours(35);
c1.addSales(400);
staffList[6] = c1;

Commission c2 = new Commission("Budi", "202 Second St.",
    "555-2222", "222-22-2222", 9.75, 0.15);
c2.addHours(40);
c2.addSales(950);
staffList[7] = c2;
```


Gambar 3. Output payroll setelah penambahan dua object Commission (Andi dan Budi)

A.6 Step 6 Buktikan Polymorphism Tanpa if/switch

```
amount = staffList[count].pay();
```

- Checkpoint 4 Aha moment: Jelaskan bagaimana reference type menentukan behavior apa yang boleh diminta melalui compiler, sedangkan actual object menentukan implementation override mana yang dijalankan saat runtime.

  Jawaban:

  Reference type (StaffMember) menentukan method apa yang boleh dipanggil, hanya yang dikenal oleh StaffMember seperti pay(); compiler akan menolak method khusus subclass seperti addHours() atau addSales() melalui reference ini. Actual object menentukan implementation override mana yang dijalankan saat runtime: baris amount = staffList[count].pay(); akan menjalankan pay() milik Executive, Employee, Hourly, Volunteer, atau Commission sesuai object yang sebenarnya.

BAGIAN B PRAKTIKUM II: PAINTING SHAPES

B.1 Step 7 Bangun Abstract Shape

Potongan kode Shape.java:

```
public abstract class Shape {
    private String shapeName;

    public Shape(String name) {
        shapeName = name;
    }

    public abstract double area();

    @Override
    public String toString() {
        return shapeName;
    }
}
```

- Coba prediksi: apakah new Shape("X") dapat dibuat?

  Jawaban:

  Tidak dapat. new Shape("X") akan menyebabkan compile error karena Shape adalah abstract class yang tidak boleh diinstansiasi langsung.

- Jelaskan mengapa area() dideklarasikan tetapi tidak diberi body di Shape.

  Jawaban:

  Karena Shape adalah bentuk yang masih umum sehingga belum memiliki rumus luas yang spesifik. Method area() dideklarasikan sebagai abstract untuk menetapkan kontrak bahwa setiap subclass konkret wajib mengimplementasikan area() dengan rumusnya masing masing.

B.2 Step 8 Implementasikan Tiga Bentuk Konkret

Potongan kode Sphere.java:

```
public class Sphere extends Shape {
    private double radius;

    public Sphere(double r) {
        super("Sphere");
        radius = r;
    }

    @Override
    public double area() {
        return 4 * Math.PI * radius * radius;
    }

    @Override
    public String toString() {
        return super.toString() + " of radius " + radius;
    }
}
```

Potongan kode Rectangle.java:

```
public class Rectangle extends Shape {
    private double length;
    private double width;

    public Rectangle(double l, double w) {
        super("Rectangle");
        length = l;
        width = w;
    }

    @Override
    public double area() {
        return length * width;
    }

    @Override
    public String toString() {
        return super.toString() + " of length " + length
               + " and width " + width;
    }
}
```

Potongan kode Cylinder.java:

```
public class Cylinder extends Shape {
    private double radius;
    private double height;

    public Cylinder(double r, double h) {
        super("Cylinder");
        radius = r;
        height = h;
    }

    @Override
    public double area() {
        return Math.PI * radius * radius * height;
    }

    @Override
    public String toString() {
        return super.toString() + " of radius " + radius
               + " and height " + height;
    }
}
```

B.3 Step 9 Lengkapi Paint.amount(Shape s)

Potongan kode Paint.java:

```
public class Paint {
    private double coverage;

    public Paint(double c) {
        coverage = c;
    }

    public double amount(Shape s) {
        System.out.println("Computing amount for " + s);
        return s.area() / coverage;
    }
}
```

- Checkpoint 5: Mengapa Paint.amount(Shape s) dapat bekerja untuk tiga subclass yang rumus area()-nya berbeda?

  Jawaban:

  Paint.amount(Shape s) hanya bergantung pada kontrak Shape.area(). Saat s.area() dipanggil, Java memilih implementasi area() milik actual object (Rectangle, Sphere, atau Cylinder) saat runtime, sehingga satu method dapat bekerja untuk tiga subclass dengan rumus area yang berbeda tanpa perlu mengetahui class konkretnya.

B.4 Step 10 Uji deck, bigBall, dan tank

Potongan kode pada PaintThings.java:

```
final double COVERAGE = 350;
Paint paint = new Paint(COVERAGE);

Rectangle deck = new Rectangle(20, 35);
Sphere bigBall = new Sphere(15);
Cylinder tank = new Cylinder(10, 30);

double deckAmt = paint.amount(deck);
double ballAmt = paint.amount(bigBall);
double tankAmt = paint.amount(tank);
```


Gambar 4. Output PaintThings.java untuk deck, bigBall, dan tank

B.5 Step 11 Uji Polymorphic Reference Secara Eksplisit

Potongan kode eksperimen pada PaintThings.java:

```
Shape s;

s = deck;
System.out.println(s + " area=" + s.area());

s = bigBall;
System.out.println(s + " area=" + s.area());

s = tank;
System.out.println(s + " area=" + s.area());
```

- Prediksi output sebelum run.

  Jawaban:

  Prediksi: baris pertama mencetak Rectangle of length 20.0 and width 35.0 area=700, baris kedua Sphere of radius 15.0 area=2827.43..., dan baris ketiga Cylinder of radius 10 and height 30 area=9424.... Hasil run sesuai prediksi Rectangle 700.0, Sphere 2827.4333882308138, Cylinder 9424.77796076938.

Gambar 5. Output eksperimen polymorphic reference Shape s

- Checkpoint 6 Aha moment: Jelaskan bagaimana satu variable s tetap bertipe Shape, tetapi method area() dapat berpindah implementasi ketika actual object yang dirujuk berubah.

  Jawaban:

  Variable s dideklarasikan bertipe Shape dan tipenya tidak pernah berubah. Tapi ketika s diarahkan ke deck, bigBall, lalu tank, pemanggilan s.area() menjalankan area() milik Rectangle, Sphere.

BAGIAN C PRAKTIKUM III: POLYMORPHIC SORTING

C.1 Step 12 Compile Numbers dan Baca Error-nya

Potongan kode awal Numbers.java:

```
int[] intList;
...
Sorting.selectionSort(intList);
```


Gambar 6. Error compile Numbers.java saat menggunakan int[]

- Simpan pesan error compiler.

  Jawaban:

  The method selectionSort(Comparable[]) in the type Sorting is not applicable for the arguments (int[]).

- Bandingkan parameter selectionSort dengan tipe int[].

  Jawaban:

  Parameter selectionSort adalah Comparable[] sedangkan argumen yang diberikan adalah int[] . Kedua tipe tersebut tidak kompatibel sehingga compile gagal.

- Checkpoint 7: Tentukan mengapa primitive int tidak memenuhi kontrak Comparable.

  Jawaban:

  int adalah tipe primitif, bukan object, sehingga tidak mengimplementasikan interface Comparable dan tidak memiliki method compareTo(). Algoritma sorting bergantung pada kontrak compareTo(), jadi int[] tidak dapat diterima sebagai Comparable[].

C.2 Step 13 – Gunakan Integer[] dan Jalankan selectionSort

Potongan kode perbaikan Numbers.java:

```
Integer[] intList;
...
intList[i] = scan.nextInt();
Sorting.selectionSort(intList);
```


Gambar 7. Output Numbers.java dengan Integer[] menggunakan selectionSort (ascending)

- Tandai di mana Sorting memanggil compareTo().

  Jawaban:

  Pada selectionSort, compareTo() dipanggil pada baris if (list[scan].compareTo(list[min]) < 0), yaitu saat membandingkan elemen saat ini dengan elemen minimum sementara.

C.3 Step 14 Gunakan Algoritma yang Sama untuk String

Potongan kode Strings.java:

```
String[] strList;
int size;

Scanner scan = new Scanner(System.in);

System.out.print("\nHow many strings do you want to sort? ");
size = scan.nextInt();
strList = new String[size];

System.out.println("\nEnter the strings...");
for (int i = 0; i < size; i++)
    strList[i] = scan.next();

Sorting.selectionSort(strList);
```


Gambar 8. Output Strings.java menggunakan selectionSort (ascending)

- Checkpoint 8: Mengapa Sorting.selectionSort(...) dapat digunakan untuk Integer[] dan String[] walaupun kedua class tersebut tidak berada dalam satu hierarchy buatan Anda?

  Jawaban:

  Karena Integer dan String sama-sama mengimplementasikan interface Comparable.

C.4 Step 15 Ubah insertionSort Menjadi Descending

Potongan kode pada Sorting.java:

```
while (position > 0 && key.compareTo(list[position - 1]) > 0) {
    list[position] = list[position - 1];
    position--;
}
```

- Prediksi terlebih dahulu perubahan kondisi compareTo() yang diperlukan.

  Jawaban:

  Prediksi: agar urutan menjadi descending, kondisi pada while diubah dari compareTo(...) < 0 menjadi compareTo(...) > 0, sehingga elemen yang lebih kecil digeser ke kanan dan key ditempatkan di depan elemen yang lebih kecil darinya. Hasil run nya Numbers menghasilkan 67 50 30 20 15 dan Strings menghasilkan Su-33 Rafale F-18 F-16 Eurofighter.

Gambar 9. Output Numbers.java menggunakan insertionSort (descending)

Gambar 10. Output Strings.java menggunakan insertionSort (descending)

C.5 Step 16 Lengkapi Salesperson.compareTo(...)

Potongan kode Salesperson.java:

```
public class Salesperson implements Comparable {
    private String firstName, lastName;
    private int totalSales;

    ...

    public int compareTo(Object other) {
        int result;
        Salesperson otherSales = (Salesperson) other;

        result = this.totalSales - otherSales.getSales();

        if (result == 0) {
            result = this.lastName.compareTo(
                        otherSales.getLastName());
            if (result == 0)
                result = this.firstName.compareTo(
                            otherSales.getFirstName());
        }

        return result;
    }
}
```

- Checkpoint 9: Comparable adalah interface. Mengapa Sorting tidak meminta Salesperson menjadi subclass khusus, melainkan cukup memenuhi contract compareTo()?

  Jawaban:

  Sorting tidak memerlukan Salesperson menjadi subclass tertentu karena Comparable adalah interface, yaitu sebuah kontrak. Selama Salesperson mengimplementasikan Comparable dan menyediakan compareTo(), Sorting dapat membandingkan dan mengurutkannya melalui reference bertipe Comparable.

C.6 Step 17 Jalankan WeeklySales

Potongan kode WeeklySales.java:

```
Salesperson[] salesStaff = new Salesperson[10];
...
Sorting.insertionSort(salesStaff);

System.out.println("\nRanking of Sales for the Week\n");
for (Salesperson s : salesStaff)
    System.out.println(s);
```


Gambar 11. Output WeeklySales.java menggunakan insertionSort (descending)

- Periksa urutan total sales dari terbesar ke terkecil.

  Jawaban:

Urutan total sales dari terbesar ke terkecil sudah benar: 7300 (Taylor), 5000 (Adams), 4935 (Duck), 3000 (Smith, Jones Jane, Jones James, Black), 2850 (Doe), 2800 (Walter), dan 1570 (Trump).

- Untuk nilai sales yang sama (3000), periksa urutan nama sesuai aturan tie-breaker dan efek sorting descending.

  Jawaban:

  Untuk sales 3000, tie-breaker pada compareTo() itu nama secara alfabet ascending. Karena insertionSort sekarang descending, urutan nama ikut terbalik: Smith, Walt, lalu Jones, Jane, lalu Jones, James, dan terakhir Black, Jane.

- Checkpoint 10 Aha moment: Jelaskan bagaimana satu class Sorting dapat bekerja pada Integer, String, dan Salesperson karena semuanya dapat diperlakukan sebagai Comparable.

  Jawaban:

  Sorting hanya memanggil compareTo() melalui reference bertipe Comparable. Integer, String, dan Salesperson sama-sama mengimplementasikan Comparable, sehingga algoritma yang sama dapat bekerja untuk ketiganya; implementasi compareTo() yang dijalankan ditentukan oleh actual object.

BAGIAN D SINTESIS HASIL PRAKTIKUM DAN TABEL PENGUJIAN

D.1 Tabel Sintesis Tiga Bentuk Polymorphism

Kasus	Reference / Abstraction	Actual Object	Message yang Sama
Employee	StaffMember	Executive / Employee / Hourly / Volunteer / Commission	pay()
Painting Shapes	Shape	Sphere / Rectangle / Cylinder	area()
Sorting	Comparable	Integer / String / Salesperson	compareTo(...)

D.2 Step 18 Refleksi Sintesis Polymorphism

1. Temukan satu baris code dari setiap kasus yang menurut Anda paling jelas menunjukkan polymorphism.

   Jawaban:

   Kasus Employee: amount = staffList[count].pay();

   Kasus Painting Shapes: return s.area() / coverage; (atau s.area() pada eksperimen Shape s).

   Kasus Sorting: if (list[scan].compareTo(list[min]) < 0) di dalam selectionSort.

2. Jelaskan mengapa menambah Commission tidak memerlukan perubahan cara loop payroll memanggil pay().

   Jawaban:

   Karena loop payroll hanya bergantung pada reference StaffMember dan method pay(). Commission adalah subclass Hourly (yang juga turunan StaffMember) dan meng-override pay(), sehingga cukup dimasukkan ke staffList tanpa mengubah baris amount = staffList[count].pay(); dynamic binding otomatis memilih pay() milik Commission.

3. Jelaskan mengapa Paint tidak perlu mengetahui rumus area setiap subtype.

   Jawaban:

   Karena Paint hanya meminta area() dari parameter bertipe Shape. Rumus luas setiap subtype disembunyikan di dalam class masing-masing (Sphere, Rectangle, Cylinder) dan dipilih otomatis saat runtime, sehingga Paint tidak perlu if/else atau instanceof.

4. Jelaskan mengapa Sorting tidak perlu mengetahui detail internal Integer, String, atau Salesperson.

   Jawaban:

   Karena Sorting hanya menggunakan kontrak compareTo() dari Comparable. Cara membandingkan (angka, urutan alfabet, atau total sales lalu nama) menjadi tanggung jawab masing-masing class, sehingga algoritma sorting tidak perlu mengetahui struktur internalnya.

D.3 Tabel Skenario Uji (T6-01 s.d. T6-09)

ID	Skenario	Expected Result	Status	Bukti Singkat
T6-01	pay() dipanggil melalui StaffMember reference	Implementation mengikuti actual subtype	PASS	Executive (2923.07), Hourly (422.0), Volunteer (Thanks!)

ID	Skenario	Expected Result	Status	Bukti Singkat
T6-02	Commission menerima hours dan sales	pay = hourly pay + commission	PASS	Andi: 298.75; Budi: 532.5
T6-03	payday() menangani subtype baru	Loop umum tidak perlu cek concrete class	PASS	Loop payday() tidak berubah, tanpa if/switch
T6-04	Paint.amount(deck/bigBall /tank)	Method yang sama memanggil area() subtype yang sesuai	PASS	Deck 2, Big Ball 8.1, Tank 26.9
T6-05	Shape reference diarahkan ke tiga object	area() berubah sesuai actual object	PASS	s.area(): 700.0, 2827.43..., 9424.77...
T6-06	int[] diberikan ke Sorting	Compile gagal karena primitive bukan Comparable object	PASS	Error: not applicable for the arguments (int[])
T6-07	Integer[] dan String[] di-sort	Keduanya dapat memakai Sorting yang sama	PASS	Numbers dan Strings terurut ascending dengan selectionSort
T6-08	Salesperson.compareTo dipakai Sorting	Urutan mengikuti sales lalu tie-breaker nama	PASS	WeeklySales terurut berdasarkan sales lalu nama
T6-09	insertionSort descending	Output Numbers, Strings, WeeklySales descending	PASS	Numbers, Strings, WeeklySales terurut descending

BAGIAN E SELF ASSESSMENT DAN REFLEKSI AKHIR

E.1 Self Assessment

Pernyataan	Skor 1-5
Saya dapat membedakan reference type dan actual object type.	4
Saya memahami mengapa overridden method dipilih saat runtime.	3
Saya dapat menggunakan superclass/abstract class sebagai polymorphic reference.	3
Saya dapat menjelaskan polymorphism melalui interface Comparable.	3
Saya dapat menambah subtype baru tanpa mengubah code umum yang hanya bergantung pada abstraction.	3

E.2 Refleksi

1. Pada loop payday(), mengapa satu pemanggilan staffList[count].pay() dapat menghasilkan perhitungan yang berbeda?

   Jawaban:

   Karena setiap elemen staffList dapat merujuk ke actual object subclass yang berbeda yang meng-override pay(), dan JVM memilih implementasinya saat runtime melalui dynamic binding. Jadi baris yang sama menghasilkan perhitungan yang berbeda sesuai tipe object sebenarnya.

2. Jika reference bertipe StaffMember menunjuk ke object Commission, apa yang ditentukan oleh reference type dan apa yang ditentukan oleh actual object?

   Jawaban:

   Reference type (StaffMember) menentukan method apa yang legal dipanggil saat compile-time (misalnya pay(), bukan addSales()). Actual object (Commission) menentukan implementasi override mana yang dieksekusi saat runtime, yaitu Commission.pay().

3. Mengapa Paint.amount(Shape s) tidak memerlukan if/else untuk membedakan Sphere, Rectangle, dan Cylinder?

   Jawaban:

   Karena Shape menjamin adanya kontrak abstract method area(), sehingga s.area() otomatis menjalankan rumus milik subclass aktual (Sphere, Rectangle, atau Cylinder) saat runtime tanpa perlu memeriksa tipe object.

4. Apa hubungan abstract method area() dengan dynamic binding pada kasus Painting Shapes?

   Jawaban:

   Abstract method area() menjadi kontrak pada compile-time untuk reference Shape, sedangkan dynamic binding mengarahkan pemanggilan tersebut ke area() milik subclass konkret yang sebenarnya dirujuk saat runtime. Tanpa abstract method tersebut, Shape tidak dapat menjamin bahwa area() bisa dipanggil melalui reference Shape.

5. Mengapa Sorting dapat mengurutkan Integer, String, dan Salesperson tanpa mengetahui struktur internal masing-masing class?

   Jawaban:

   Karena Sorting hanya bergantung pada kontrak interface Comparable melalui pemanggilan compareTo(), tanpa peduli atribut internal apa yang dibandingkan di dalam masing-masing class.

6. Mengapa int[] gagal digunakan pada Sorting tetapi Integer[] berhasil?

   Jawaban:

   Karena int adalah tipe primitif yang bukan object dan tidak mengimplementasikan Comparable, sedangkan Integer adalah wrapper class (object) yang mengimplementasikan Comparable sehingga memenuhi parameter Comparable[].

7. Apa perbedaan polymorphism via inheritance pada StaffMember/Shape dengan polymorphism via interface pada Comparable?

   Jawaban:

   Polymorphism via inheritance (StaffMember/Shape) menyatukan class dalam satu hierarchy keturunan yang mewarisi state dan behavior dari superclass. Polymorphism via interface (Comparable) dapat menyatukan class dari hierarchy yang berbeda (Integer, String, Salesperson) selama memenuhi kontrak method yang sama, tanpa pewarisan implementasi.

8. Lengkapi dengan bahasa Anda sendiri: "Polymorphism bukan berarti object berubah menjadi object lain, tetapi ..."

   Jawaban:

   ... kemampuan satu reference atau kontrak abstraksi yang sama untuk merujuk ke berbagai tipe object konkret yang berbeda, di mana pemanggilan method yang sama akan mengeksekusi implementasi yang sesuai dengan tipe object aktualnya pada saat runtime.